Route and prompt sheet (hackathon scope)

Single source of truth for: the master pre-prompt, the route, the `deck.json` shape, the build prompts (what we give the coding AI) and the runtime prompts (what the app gives the model). Scope comes from `feature-spec.md`; the user and demo from `problem-statement.md`; layouts from `wireframes.html`.

Part A. Master pre-prompt (paste this before ANY build prompt)

```
PROJECT CONTEXT - read fully before writing any code.

WHAT: You are building [APP NAME TBD], a mobile-first web app for a hackathon, in the public GitHub
repo Bhuvaneshwer2005/Bits-and-Bytes. It turns a student's notes PDF into a spoken, captioned
flashcard deck, then a quiz on what they just studied, then a one-page cheat sheet, and rewards daily
use with a streak and a calendar heatmap. There is NO chat window anywhere: the interface is cards,
voice, captions, a quiz and a heatmap. The user is a first- or second-year student studying in short
gaps (bus, queue) on a phone. English only.

JUDGING (design every decision against this): user experience 25 (few steps, clear flow, real error
and empty states; also the tie-break), visual design 20 (layout, type, colour, consistency),
accessibility 15 (contrast, text size, alt text, 44px tap targets, captions, keyboard, reduced
motion), responsible use of AI 15 (our submitted prompts + what we changed by hand), working
prototype 15 (a live HTTPS link must work end to end on a phone), presentation 10 (3-minute demo).
A late submission scores 0, so the app must be deployable after every step.

HOW IT WORKS: PDF text is extracted in the browser (pdf.js). One small serverless function, /api/ai,
holds the API key (from env vars only), rate-limits, and calls the model (provider swappable). It
runs staged prompts that return JSON only: P1 outline -> P2 cards per section -> P3 grounding check
-> P4 quiz per section -> P5 cheat-sheet summary; P6 regenerates one card on demand. Code validates
every response against a zod schema (one retry, then fall back to the bundled sample deck with a
visible notice), verifies in code that each card's source_quote really appears in that page's text,
and assembles everything into one deck.json (schema: Part C, the single source of truth), cached in
localStorage by file hash. Everything else renders locally from deck.json: the flashcard player, TTS
via the Web Speech API (highlight captions sentence by sentence, because word boundaries are
unreliable on phones), a quiz graded locally with seeded shuffled options, streak + heatmap from a
localStorage daily log, and the cheat-sheet PDF via pdfmake. No accounts. PDFs are never stored on
a server.

SCOPE: Must: upload (drag and drop, page range, validation, scanned-PDF error), deck JSON,
flashcards with voice and captions, quiz, streak badge + daily goal, calendar heatmap (the demo shows
clearly labelled SAMPLE history), AI transparency (an "AI-generated" label, source page on every card
and question, a "check this" flag when in_source is false, a privacy note, a hand-edit log in
docs/ai-log.md), and a bundled sample deck as the demo fallback. Should: one cheat-sheet PDF; edit or
regenerate a card. Cut: XP, streak freeze, extra PDF templates, diagrams, MP4/PPTX, OCR, accounts,
other languages.

CONVENTIONS: React + Vite + TypeScript unless told otherwise. Design tokens as CSS variables (light
and dark), one primary action per screen, mobile-first at 360px with no horizontal scroll, layout
reference in docs/wireframes.html. Never commit secrets (the repo is public); use env vars. AI output
is data and is never rendered as HTML. Streak days use the device's LOCAL calendar day; a day counts
after 10 cards plus a finished quiz. Commit once per build step and tag it (v0.1.0, v0.2.0, ...) with
a tag message listing the changes. Record every hand edit to AI output in docs/ai-log.md.

HOW YOU WILL BE TASKED: one build step at a time (B0 to B9), each stating its PREVIOUS step (what
already exists), CURRENT task (what to build), and NEXT step (what it will expect from you). Do only
the current step, leave clean seams for the next, and finish by listing what you changed and how to
verify it.
```

Part B. Route

Code steps (C) do the cheap, predictable work. The AI is called only where language understanding is needed, and it only ever writes JSON.

```
C1 Upload + extract text (code, in browser)
 -> P1 Outline -> P2 Cards (per section) -> P3 Grounding check (per section)
 -> P4 Quiz (per section) -> P5 Cheat-sheet summary
 -> C2 Assemble + validate deck.json, cache by file hash
 -> C3 Player: flashcards + TTS + captions
 -> C4 Quiz screen (graded locally, only cards the user viewed)
 -> C5 Streak + daily goal + heatmap (localStorage log)
 -> C6 Cheat-sheet PDF (from deck.json)

On demand: P6 Regenerate one card -> back through P3 -> its quiz question rebuilt by P4
```

Sections run P2, P3, P4 in parallel. The first finished section streams into the UI, so the first card plays well before the whole deck is done.

Build step	Runtime steps it adds	What the user sees	Tag
B0 Skeleton + live deploy	none	Home screen shell, design tokens, sample deck opens, live URL	v0.1.0
B1 Upload + outline	C1, P1	Drop a PDF, pick pages, progress, outline	v0.2.0
B2 Cards + checks	P2, P3, C2	Deck overview, AI label, source pages, flagged cards	v0.3.0
B3 Player	C3	Flashcards with voice and captions	v0.4.0
B4 Quiz	P4, C4	Quiz after the deck, results	v0.5.0
B5 Streak + heatmap	C5	Streak badge, daily goal, calendar heatmap	v0.6.0
B6 Cheat sheet	P5, C6	One-page PDF	v0.7.0
B7 Edit + AI log	P6	Edit or regenerate a card, hand-edit log	v0.8.0
B8 States + accessibility	none	Error and empty states, a11y pass, polish	v0.9.0
B9 Demo hardening	none	Fallbacks proven, rate limit, rehearsal, submission pack	v1.0.0

Every AI call has a fallback: one retry, then the bundled sample deck with a visible notice saying so.

Part C. deck.json (canonical shape)

```
meta     { title, source_file, file_hash, pages, model, created, ai_generated: true }
sections [{ id, title, page_range:[from,to], summary }]
cards    [{ id, section_id, type: term|qa|cloze, front, back,
            say_front, say_back,           // spoken text; captions are split from this in code
            source_page, source_quote,     // short exact quote from that page of the PDF
            quote_found: true|false,       // set by CODE (string match), not by the model
            in_source: true|false,         // set by P3 (and forced false if quote_found is false)
            check_reason,                  // P3's one-line reason when in_source is false
            edited_by_user: false }]
quiz     [{ id, card_id, type: mcq|tf|cloze, prompt, options, answer_index, explanation, source_page }]
summary  { bullets_by_section: [{ section_id, bullets }], key_terms: [{ term, meaning }] }
```

Local-only data (not in deck.json): `daily_log {"YYYY-MM-DD": {cards, quiz_done}}`, `settings {goal, voice, rate, caption_size}`, `viewed_card_ids`, `again_card_ids`.

Part D. Build prompts (give the Master pre-prompt first, then one of these)

Each one names the step before it, the step it builds, and the step after it. Acceptance criteria are what we check before tagging.

B0. Skeleton + live deploy

- **Previous:** nothing; the repo only has docs/.
- **Next:** B1 adds the real upload screen and needs routing, tokens and the `/api/ai` stub in place.

```
Create the Vite + React + TypeScript app at the repo root. Add: CSS design tokens (colour, type
scale, spacing, radius; light and dark; one accent) and a base layout that is mobile-first at 360px;
a router with screens Home, Build, DeckOverview, Player, Quiz, Results; the Home screen from
wireframe 1 (streak pill and daily-goal bar with static placeholders, drop zone that does nothing
yet, "Try the sample deck" button, privacy note); a bundled sample deck at src/data/sample-deck.json
matching Part C (about 12 cards over 2 sections, sample content is fine for now); a serverless
function api/health returning {ok:true} and a stub api/ai that returns 501; .env.example listing
AI_PROVIDER and AI_API_KEY; .gitignore covering .env; docs/ai-log.md with the table header.
Deploy to Vercel (or Netlify) connected to the GitHub repo and confirm the live HTTPS URL opens on
a phone width.
Acceptance: live URL works on a phone; "Try the sample deck" opens DeckOverview from the sample
JSON; no secrets in git; lint and build pass.
```

B1. Upload + outline (P1)

- **Previous:** B0 gave the shell, tokens, router and stubbed `/api/ai`.
- **Next:** B2 needs, per section, the page text and page range, and a working model call.

```
Implement the upload flow (wireframes 1 and 2). Drag-and-drop plus click-to-browse; accept PDF
only, max 20 MB and 60 pages, with friendly errors. Extract text per page in the browser with
pdf.js; if a PDF yields almost no text, show the scanned-PDF error (wireframe 8) instead of trying
OCR. Compute a SHA-256 file hash. Page-range picker defaulting to all pages (cap 40 pages used).
Implement api/ai: reads AI_PROVIDER and AI_API_KEY from env, accepts {promptId, system, user}, calls
the model, returns the raw JSON text, rate-limits per IP, and never logs the key or the user's PDF
text. Implement a client helper callAI(promptId, inputs) that validates the response against a zod
schema, retries once on invalid JSON, and on a second failure throws a typed AiFailure so the UI
can fall back. Wire prompt P1 (Part E) and show the Build screen's named stages with real progress,
then the section outline. Recent-decks list reads localStorage by file hash and reuses a cached
deck.
Acceptance: a text PDF produces a 3-8 section outline with page ranges that cover the chosen pages;
a scanned or oversized file shows the right message; uploading the same file twice skips the AI;
the live deploy works with the key set only in the host's env settings.
```

B2. Cards + grounding checks (P2, P3, C2)

- **Previous:** B1 produced a validated outline and per-page text.
- **Next:** B3 plays `cards[]` and needs `say_front` and `say_back`; B4 needs `in_source` set on every card.

```
For each section, in parallel, run P2 then P3 (Part E). After P2, check in code that each
source_quote appears in that page's text (normalise whitespace and case); set quote_found; if false,
force in_source false before P3. After P3 merge results into cards[]. Assemble and validate the full
deck.json (Part C), cache it in localStorage by file hash, and stream each finished section into
DeckOverview. Build DeckOverview per wireframe 3: title, an "AI-generated, check it against your
notes" label, sections with card counts and a flagged count, a "check this" marker on cards where
in_source is false, and Start listening / Cheat sheet buttons (cheat sheet disabled until B6).
Acceptance: the first section appears before the last finishes; every card has a source page; cards
with a missing quote or in_source false are visibly flagged; a deliberately bad model response falls
back to the sample deck with a notice.
```

B3. Player with voice and captions (C3)

- **Previous:** B2 produced a validated `deck.json` with flagged cards.
- **Next:** B4 needs the set of cards the user actually viewed, saved as `viewed_card_ids`, and `again_card_ids`.

```
Build the Player (wireframe 4). Card flips on tap, space, or the flip button; arrows move between
cards. Auto-play mode uses the Web Speech API: read say_front, short pause, flip, read say_back,
advance. Controls: play/pause, previous/next, repeat, speed 0.75-1.5x, voice picker from the
device's English voices, captions on/off, caption size (3 steps). Split say_front/say_back into
sentences in code and highlight the current sentence in the caption bar. Progress bar and section
name. "Again" re-queues the card at the end; "Got it" does not. Show source page on every card and
an "AI-generated" note; flagged cards show the check-this marker. Persist position and settings in
localStorage; record viewed_card_ids as cards are shown. Respect prefers-reduced-motion (no flip
animation). Handle no-voices-available with a visible captions-only mode.
Acceptance: works on a phone browser with sound; captions stay in sync with speech; every control is
at least 44px; keyboard-only use works; reduced motion removes the animation.
```

B4. Quiz (P4, C4)

- **Previous:** B3 saves `viewed_card_ids` and `again_card_ids`.
- **Next:** B5 needs a `quiz_done` event with the score, written to the daily log.

```
Run P4 per section (after P3, in the background while the user studies) and validate against the
schema; drop any bad question and never retry the whole quiz for one bad question. Quiz screen
(wireframe 5): only questions whose card_id is in viewed_card_ids, shuffle option order with a
seeded shuffle, grade locally, show an icon plus text for right and wrong (not colour alone), show
the explanation and source page after each answer. Results (wireframe 6, without the heatmap yet):
score, weakest section, "Review N missed cards" that sends those cards back to the Player, and
Download cheat sheet (disabled until B6). Emit quiz_done {score, date}.
Acceptance: no question for a card the user hasn't seen; the correct answer is not always in one
slot; a dropped question leaves the quiz working; all feedback is understandable without colour.
```

B5. Streak, daily goal and heatmap (C5)

- **Previous:** B3 and B4 emit card views and `quiz_done`.
- **Next:** B6 and B7 don't depend on it, but B8's accessibility pass will test the heatmap's labels.

```
Add the daily log in localStorage keyed by local date (YYYY-MM-DD, device local day, never UTC).
Increment cards when a card is completed and set quiz_done when a quiz finishes. A day counts for the
streak when cards >= 10 and quiz_done. Streak rules: same day changes nothing, next day adds 1, a gap
of 2+ days restarts at 1, a clock moved backwards changes nothing. Guard every storage read and write
with try/catch and a working fallback. Home shows the streak pill and the daily-goal bar (goal
5/10/20, default 10). Results shows the 5-week heatmap (wireframe 6): each cell shows its number, has
an aria-label like "12 March: 9 cards", and is not colour-only. Add a "Load sample history" toggle
that fills the log with clearly labelled SAMPLE data for the demo and can be removed.
Acceptance: unit tests cover the streak rules (same day, next day, month/year end, gap, clock
backwards, corrupt storage); the heatmap is readable by a screen reader; sample data is visibly
labelled.
```

B6. Cheat sheet (P5, C6)

- **Previous:** B2 and B4 produced verified cards and the quiz.
- **Next:** B7 may change cards; the PDF must always be built from the current deck.json.

```
Run P5 when the deck is ready and validate it. Render a one-page A4 cheat sheet with pdfmake from
deck.json only (no new AI call at export): title, AI-generated note, bullets per section, key terms,
footer with source file name and date. If content would overflow one page, shrink in this order:
fewer key terms, shorter bullets, smaller type. Wire the Download buttons on DeckOverview and
Results. Use only verified cards (in_source true).
Acceptance: always exactly one page; text is selectable; works offline once the deck exists; the PDF
reflects the current deck after edits.
```

B7. Edit and regenerate a card (P6) + AI log

- **Previous:** B2 to B6 are complete.
- **Next:** B8 will test this screen's labels and states.

```
Build the Edit screen (wireframe 7): editable front and back, shows the source page and quote, a
"Regenerate with AI" action with an optional note that calls P6, then re-runs the code quote check
and P3, then rebuilds that card's quiz question with P4. Mark edited_by_user true and show an
"Edited by you" badge on the card. Add Delete card. Every manual change appends to a local
hand-edit list exportable as text so we can paste it into docs/ai-log.md. Keep the cached deck in
sync and invalidate the cheat sheet.
Acceptance: an edit survives a reload; regenerating a card re-checks it; a deleted card disappears
from the player and the quiz.
```

B8. States, accessibility and polish

- **Previous:** all features exist.
- **Next:** B9 rehearses the demo, so nothing visible may still be a placeholder.

```
Implement every error and empty state in wireframe 8: no decks yet, scanned PDF, too large, AI
unavailable (fall back to the sample deck with a notice), no voices on the device, offline. Run an
accessibility pass: text and UI contrast of at least 4.5:1 (3:1 for large text), visible focus rings,
tap targets of at least 44px, alt text or aria-labels for all icons, headings in order, captions
resizable, prefers-reduced-motion honoured, full keyboard use, and test at 200% text zoom with no
clipping. Check consistency against the tokens (no one-off colours or sizes). Remove all placeholder
text.
Acceptance: Lighthouse accessibility 95+, no horizontal scroll at 360px, every screen reachable and
usable by keyboard.
```

B9. Demo hardening and submission pack

- **Previous:** B8 produced the polished app.
- **Next:** nothing; this is the final tag before the 2:40 deadline.

```
Prove the fallbacks by simulating: key removed, model slow, invalid JSON, offline. Set a rate limit
and a spend cap on /api/ai. Make the sample deck the real demo deck (about 25 cards from the chosen
chapter) with its pre-generated quiz and cheat sheet. Write the submission pack in docs/submission.md:
live link, libraries list with licences, the prompts (Parts A, D and E), and the completed
docs/ai-log.md. Rehearse the 3-minute demo from problem-statement.md and fix anything that stumbles.
Tag v1.0.0.
Acceptance: the live link works on a phone on mobile data; the demo completes in under 3 minutes
using only the sample deck path and again using a fresh PDF; nothing in the repo contains a key.
```

Part E. Runtime prompts (what the app sends the model)

Every runtime call uses a fixed system message (the block below, filled in by the app) plus a user message holding the inputs. Because each carries the previous, current and next step, the model knows what its output is for.

```
[ROUTE CONTEXT]
App: a study app that turns a student's notes PDF into spoken flashcards, a quiz and a cheat sheet.
Step {{n}} of 6: {{current_name}}
Previous step ({{prev_name}}): {{prev_summary}}
This step: {{current_goal}}
Next step ({{next_name}}): will consume {{next_needs}}
Rules: Output JSON only, exactly matching the schema; any extra text breaks the pipeline. Use ONLY
the supplied PDF text. Never invent facts; if something isn't in the text, say so (in_source:false).
Write plain English a first-year student understands. Your output is shown to a student as
AI-generated, so be accurate and conservative.
```

P1. Outline

- **Previous:** C1 extracted the text, with page numbers and heading candidates.
- **Next:** P2 needs sections with page ranges so it can build cards one section at a time.
- **App supplies:** page texts (trimmed to about 600 characters each for this call) and any detected headings.
- **Code checks:** 3-8 sections, ranges cover the chosen pages without gaps or overlaps.

```
Group the pages into 3-8 sections. For each: a short title (max 6 words), a one-sentence summary, and
the page range it covers. Every page belongs to exactly one section; follow the document's own
headings where they exist.
Output: {"sections":[{"id":"s1","title":"","summary":"","page_range":[1,3]}]}
```

P2. Cards (per section)

- **Previous:** P1 produced sections with page ranges.
- **Next:** C2 checks each source_quote in code, then P3 checks each card's meaning against its quote.
- **App supplies:** the section title and the full text of its pages, with page numbers.
- **Code checks:** 8-12 cards, fronts under 15 words, backs under 35, source_quote found in the page text.

```
For section {{section_id}} ({{title}}), using only the text of pages {{range}}, write {{n}} flashcards
(default 8-12). Types: term (word -> meaning), qa (question -> answer), cloze (sentence with one blank).
Each card tests ONE idea. Keep fronts under 15 words and backs under 35. Also write say_front and
say_back: the same content as natural spoken English (spell out symbols, formulas and abbreviations
so a voice can read them). Add source_page and a short exact source_quote copied character for
character from that page.
Output: {"cards":[{"id":"","section_id":"","type":"","front":"","back":"","say_front":"","say_back":"","
    source_page":0,"source_quote":""}]}
```

P3. Grounding check (per section)

- **Previous:** P2 produced cards with source quotes (already string-checked by code).
- **Next:** P4 builds quiz questions only from cards with in_source true, and the app flags the rest.
- **App supplies:** the cards and the text of their source pages.
- **Code checks:** one result per card_id; a card whose quote was not found stays false regardless.

```
For each card, decide whether its back is fully supported by its source_quote and the page text. Mark
in_source true or false. If false, give a one-line reason. Do not rewrite cards. Be strict: a claim that
goes beyond the quote, adds an example the page doesn't contain, or changes a number is false.
Output: {"checks":[{"card_id":"","in_source":true,"reason":""}]}
```

P4. Quiz (per section)

- **Previous:** P3 marked which cards the PDF supports.
- **Next:** C4 shows only questions for cards the user viewed, grades locally, and needs exactly one correct answer index.
- **App supplies:** cards of this section with in_source true.
- **Code checks:** valid card_id, 4 unique options for mcq, answer_index 0-3, no "all/none of the above"; bad questions are dropped, never retried alone.

```
Using ONLY the supplied cards, write {{count}} questions (default 3 per section), each tied to a
    card_id,
mixing mcq (4 options), tf and cloze. MCQ distractors must be plausible misconceptions from the same
section, never silly or obviously wrong. Add a one-sentence explanation and the source_page. Do not
place the correct answer in a fixed position; the app shuffles options afterwards.
Output: {"quiz":[{"id":"","card_id":"","type":"","prompt":"","options":[],"answer_index":0,"explanation"
    :"","source_page":0}]}
```

P5. Summary / cheat sheet

- **Previous:** P4 produced the quiz from verified cards.
- **Next:** C6 lays this out as a one-page PDF, so it must be short enough to fit.
- **App supplies:** all cards with in_source true, grouped by section.
- **Code checks:** at most 4 bullets per section, bullets at most 18 words, at most 10 key terms.

```
From the verified cards only, write a one-page cheat sheet: for each section, up to 4 bullets of at most
18 words, then up to 10 key terms across the deck, each with a meaning of at most 12 words. No new facts
beyond the cards.
Output: {"summary":{"bullets_by_section":[{"section_id":"","bullets":[]}],"key_terms":[{"term":"","meani
    ng":""}]}}
```

P6. Regenerate one card (on demand)

- **Previous:** P2 originally wrote this card and the user asked for a better version.
- **Next:** the code quote check and P3 re-check the new card, then P4 rewrites its quiz question.
- **App supplies:** the old card, the text of its source page, and the user's note.
- **Code checks:** same as P2 for a single card; the id stays the same.

```
Rewrite card {{card_id}} using only page {{source_page}}. The user says: "{{user_note}}". Keep the same
type unless the note asks otherwise. Same rules and schema as P2, one card, same id.
Output: {"card":{...}}
```

Part F. Responsible-AI log (rubric: "your submitted prompts + what you changed by hand")

Lives in `docs/ai-log.md`, created in B0 and updated at every step. It is submitted with Parts A, D and E.

Date	Step	Prompt used (id)	What the AI produced	What we changed by hand and why

The app mirrors this for users: an "AI-generated" label on every deck, the source page on every card and question, a "check this" flag when `in_source` is false, and `edited_by_user` marking cards a person changed.

Part G. Open items

1. Which model and provider (the `/api/ai` wrapper keeps it swappable).
2. Sample deck chapter for the demo.
3. Whether P3 uses the same model or a cheaper one.
4. App name.